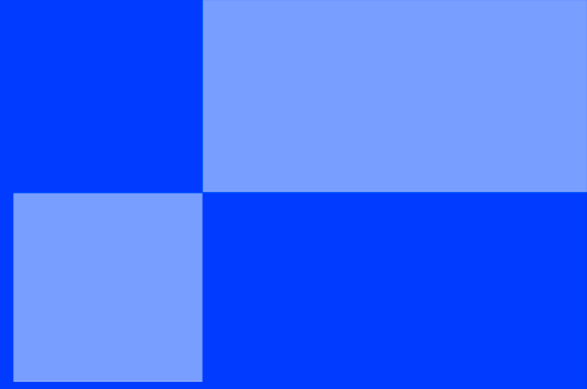




DAY 06 - BONUS TASKS

< ADVANCED />

DAY 06 - BONUS TASKS

BONUS: PUTTING IT ALL TOGETHER



Now that you have mastered user-defined functions, recursion, and built-in functions, let's combine them into practical challenges that fit right into your current skill set.

1. Mixed Challenges

Task 1.1

Write a function `analyze_grades(grades)` that takes a list of numbers representing student scores. Using the built-in functions you learned today, the function must calculate and print:

- The highest grade in the class.
- The lowest grade in the class.
- The average grade.



You don't need a `for` loop to find the average! Use a combination of `sum()` and the `len()` function.

Task 1.2

Write a **recursive** function `reverse_string(text)` that takes a string of characters and returns it completely backwards.



`reverse_string("Epitech")` should return `"hcetipE"`.



Think about the base case: what should the function return if the string is empty? For the recursive step, try separating the very first character from the rest of the string.

Task 1.3

Let's revisit the kitchen from the previous tasks. Write a function `make_custom_sandwich(ingredients)` that takes a list of strings, for example: `["bread", "lettuce", "tomato", "ham", "bread"]`.

Before making the sandwich, your function must check if it is valid:

- Use the `any()` built-in function to verify if either `"ham"` or `"tomato"` is present in the list.
- Check if the list contains at least two `"bread"` elements. If not, print `"Error: A sandwich needs top and bottom bread!"`.
- If the list is valid, print each ingredient line by line to display your creation.

2. Algorithmic Thinking

Task 2.1

Write a function `find_longest_word(words)` that takes a list of strings and returns the longest word. If there are multiple words with the same maximum length, return the first one you encounter.



`find_longest_word(["apple", "banana", "cherry", "kiwi"])` should return `"banana"`.



You can try to solve this using a standard `for` loop, or look into how the `max()` built-in function can take a `key` parameter (like `key=len`).

Task 2.2

Write a **recursive** function `count_digits(n)` that takes a positive integer and returns the total number of digits it contains.



`count_digits(42000)` should return `5`.



You are **not** allowed to convert the number into a string (no `len(str(n))`)! Instead, use mathematics. If you divide a number by 10 (using integer division `//`), you remove its last digit. What is the base case?

v1

{EPITECH}